

```
# data handling
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder
from sklearn.model_selection import train_test_split

#max_iter->no of epochs
```

## ✓ Numpy Important \*\*

```
#numpy important

a=np.array([1,2,3]) #specify list ,list(list) or scaler

b=np.zeros((3,4)) #specify shape (row,col)

c=np.ones((2,2)) #specify shape (row,col)

e=np.random.rand(3,4) #specify shape (row,col) val between(0,1)

f=np.random.randn(3,4) #specify shape (row,col) val between(-INF,+INF) NORMAL DISTRIBUTION

g=b.T #transpose

h=e.flatten() # nd array to 1d array

adotb=np.dot(a,b) #if a(MxN) and b(NxP) matrix ,then for a.b no of col of A=no of row of B
```

## ✓ Pandas Important

```
#pandas important
df=pd.read_csv("file path")
df.head()
df.info()
df.describe()
df.isna().sum()
df.fillna(df.mean())    #use mean when distrbution is normal , if skewwd use median >only when
df.dropna()
df.drop(columns=["col name"])
df.corr()
#use lable encoder for transforimg cotegorical col into numerical col
#df.iloc[row_index, column_index]

le = LabelEncoder()
df['target'] = le.fit_transform(df['target'])
X=df.drop("target",axis=1)
y=df["target"]
```

## ✓ **Matplotlib Important**

```
# import matplotlib.pyplot as plt
# plt.scatter(x,y)
# plt.plot(x,y)
# plt.xlabel("xlabel")
# plt.ylabel("ylabel")
```

## ✓ **sklearn scaling standardisation/normramlisation**

```
#sklearn important for scaling
#use when data is normal
scaler = StandardScaler()
X_trainscaled = scaler.fit_transform(X)
x_testsclaed=scaler.transform(x_test)

#use whne we know min and max of given quantity
scaler = MinMaxScaler()
X_trainscaled = scaler.fit_transform(X)
x_testsclaed=scaler.transform(x_test)

#use robust scaler if there are outliers present
```

## ✓ **HyperParameter Tuning using gridsearchcv and randomsearchv**

```
from sklearn.model_selection import GridSearchCV,RandomizedSearchCV
from sklearn.linear_model import LogisticRegression
lr=LogisticRegression()
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l2'],
    'solver': ['lbfgs']
}
grid = GridSearchCV(
    estimator=lr,
    param_grid=param_grid,

)
grid.fit(x_train,y_train)
grid.best_params_
grid.best_score_
```

## ✓ **Important Skelarn Imports**

```
#sklearn import models
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import Ridge,Lasso
from sklearn.svm import SVC,SVR
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.ensemble import RandomForestClassifier,RandomForestRegressor
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import GradientBoostingClassifier, GradientBoostingRegressor
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
```

## ✓ creating python virtual env

```
#creating virtual env in python
python -m venv venv
venv\Scripts\activate
```

## ✓ Outliers

```
#points which show abnormal behaaviour
#they affect weight based and linear models
#methods of handling outliers ->1.trimming
                        # 2.capping (set limits)
                        # 3.discretization
                        # 4.treat as missing values and impute

#to see distribuion of data
#use plt.hist(col[i],bins=30) or plt.boxplot(col[i]) to check for outliers
#for detecting outliers -> 1.z score
                        # 2.iqr
                        # 3.percentile
                        # 4.winsorization

#using z score ->the data should be normallly distributued
#if a point is greater than mean+3*sd or less u-3*sd then it can be considered as a outlier
#do z socre on that col and remove or cap those values which are >3 or <-3
#using iqr
# 75percentile=col.quantile(0.75)
# 25percentile=col.quantile(0.25)
# iqr=75percentile-25percentile
# upper_bound=75percentile+1.5*iqr
# lower_bound=25percentile-1.5*iqr
#remove or cap those values which are >upper_bound or <-lower_bound
```

## ✓ Exploratory Data Analysis

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
#data ->numerical,categorical
#df=pd.read_csv("train.csv")
tip=sns.load_dataset("tips")
#df.head()
# x=df.drop("Survived",axis=1)
# y=df["Survived"]

#univariate analysis
#for categorical data

#1.count plot
#sns.countplot(x=df["Survived"]) #df['Survived'].value_counts()

#2.pie plot
#df["Survived"].value_counts().plot(kind="pie",autopct="%1.2f%%")

#for numerical data

#1.histogram
#plt.hist(df["Age"])

#2.distplot
#sns.distplot(df["Age"])

#3.box plot
#sns.boxplot(x=df["Age"])

#bivaritate/multivariate ananlysis

#1.numerical -numerical
#plt.scatter(x=tip["tip"], y=tip["total_bill"])
# plt.xlabel("Tip")
# plt.ylabel("Total Bill")
#sns.scatterplot(x=tip["total_bill"], y=tip["tip"])
#sns.scatterplot(x=tip["total_bill"], y=tip["tip"],hue=tip['sex']) multivariate anayliss

#2.numerical-categorical
#sns.barplot(x=tip["day"],y=tip["total_bill"])

```

## ~ Handling Imbalanced Dataset

```

#if there is imbalance is dataset, it causes the model to be biased towrads to majoruty class

#methods to achive balanced dataset
# 1.undersampling>sample out majority class samples equal to majority class samples
# 2.oversampling ->duplicating minor class samples with duplication
#3.smote->over sample minor class using knn by creating new data poinst

```

```

from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import RandomOverSampler
from imblearn.over_sampling import SMOTE
rus=RandomUnderSampler()
ros=RandomOverSampler()
smote=SMOTE()
x_train_balanced,y_train_balanced=rus.fit_resample(x_train,y_train)
x_train_balanced,y_train_balanced=ros.fit_resample(x_train,y_train)
x_train_balanced,y_train_balanced=smote.fit_resample(x_train,y_train)

```

## ✓ Principle Component Analysis

```

#Principle Component Analysis (used for feature extraction)
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

#bring higher dimensional data into lower dimensional data with conserving data patterns/information
#easy to visualize the data
#here no of principle components <= no of input features
#it used covariance matrix for finding the eigen vector and values
#covariance gives the relation between two features
#no of eigen vector=no of features
#scaling is very important in pca for better results
#random state is used so that the results are reproducible
df=pd.read_csv("mnist_train.csv")
df.describe()
df.shape
X=df.iloc[:,1:]
Y=df.iloc[:,0]
x_train,x_test,y_train,y_test=train_test_split(X,Y,test_size=0.2,random_state=42)
scaler=StandardScaler()
scaler.fit(x_train)
x_train_scaled=scaler.transform(x_train)
x_test_scaled=scaler.transform(x_test)

#pca=PCA(n_components=0.95) or pca=PCA(n_components=50)
pca=PCA(n_components=50)

pca.fit(x_train_scaled)
x_train_transformed=pca.transform(x_train_scaled)
x_test_transformed=pca.transform(x_test_scaled)

#now this x_train_transformed and x_test_transformed are ready to use

#eigen values
pca.explained_variance_

```

```
#eigen vectors  
pca.components_  
  
#when we want to take optimal number of principle components  
#we specifit n_components=0.95 meaning take those eigen vectors whose variance sum upto >=95%
```

```
(48000, 784)
```

## ✓ Simple Linear Regression

```

#simple linear regression (one input feature)
#it tries to find the best fit line
#it uses ols(closed expression to calculate weights and bias)

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

df=pd.read_csv("placement.csv")
X=df.iloc[:,0]
Y=df.iloc[:,1]
X=X.values.reshape(200,1)
Y=Y.values.reshape(200,1)

x_train,x_test,y_train,y_test=train_test_split(X,Y,test_size=0.2,random_state=2)
sd=StandardScaler()
X_train_scaled=sd.fit_transform(x_train)
x_test_scaled=sd.transform(x_test)
x_test=x_test.reshape(40,1)
lr=LinearRegression()
lr.fit(X_train_scaled,y_train)
#input=np.array(4)
#input=input.reshape(1,1)
lr.predict(x_test)

#weights =lr.coef_
#bais=lr.intercepts

```

## ✓ Regression Metrics

```

from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score,root_mean_squared_er

# 1.mean absolute error
# 2.mean squared error #not robust to outliers
# 3.root mean squared error #not robust to outliers
# 4.r2 ->gives goodnes of fit of the model

mean_absolute_error([1,1],[2,1])
mean_squared_error([1,1],[2,1])
r2_score([1,1],[2,1])
root_mean_squared_error([1,1],[2,1])

```

0.7071067811865476

## ✓ Multiple Linear Regression

```
#works on more than one input features
#y=m1x1+m2x2+b
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
lr=LinearRegression()
lr.fit(x_train,y_train)
lr.predict(x_test)
```

## ✓ Stochastic Gradient Descent Regressor

```
#based on gradient descent algo
# 1.batch gradient descent->uses entire dataset to update weights
# 2.stochastic gradient descent->uses each data point to update weights
# 3.mini batch -> uses small batch from entire dataset to update weights

#when the model reaches the convergence point , it has spikes ,so avoid it uses learning rate sc

sgd=SGDRegressor(penalty='l2') # using ridge regression
sgd=SGDRegressor(penalty='l1') #using lasso regression
sgd=SGDRegressor(penalty='ElasticNet') #using elasticnet regression
sgd.fit(x_train,y_train)
sgd.predict(x_test)
```

## ✓ Polynomial Regression

```
#USED WHEN DATA IS NOT LINEAR
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
pr= PolynomialFeatures(degree=2) #here degree is an hyperparameter
x_train_transformed=pr.fit_transform(x_train)
x_test_transformed=pr.transform(x_test)
lr=LinearRegression()
lr.fit(x_train_transformed,y_train)
lr.predict(x_test_transformed)
```

## ✓ Bias variance Tradeoff

```
bias->inability of model to learn from data
variance->model behaves diff for diff data

high variance ,low bias-> overfitting
high bias ,low variance ->underfitting

#best model is low variance , low to high bias

#methods to solve overfitting -
1.regularization
```



- 2. bagging
- 3. boosting

## ✓ Ridge Regularization (L2)

```
#it has alpha as hyperparameter ( 0<= alpha<=INF)
#here loss function+alpha*(weight)^2
#weights decrease as alpha increases
#very high alpha leads to underfitting
#to be used when no of input features>=2 and all input feature are relevant
#solution obtained when contour plot of loss function and ridge expression meet
from sklearn.datasets import load_diabetes
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
data=load_diabetes()
x=data.data
y=data.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
ridge=Ridge(alpha=0.1)
ridge.fit(x_train,y_train)
y_pred=ridge.predict(x_test)
r2_score(y_test,y_pred)
```

## ✓ Lasso Regression(L1)

```
#it has alpha as hyperparameter ( 0<= alpha<=INF)
#here loss function+alpha*abs(weight)
#to be used when no of input features>=2 and all input feature are not relevant
#very high alpha leads to some weights being zero(sparcity) because alpha term comes in numerator
from sklearn.datasets import load_diabetes
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
data=load_diabetes()
x=data.data
y=data.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
Lasso=Lasso(alpha=0.1)
Lasso.fit(x_train,y_train)
y_pred=Lasso.predict(x_test)
r2_score(y_test,y_pred)
```

## ✓ ElasticNet Regression

```
#combination of both ridge and lasso regression
#here loss function+a*weight^2+b*abs(weight)
#here alpha= a+b
#l_ratio=a/alpha
```

```
#used when high input features
from sklearn.linear_model import ElasticNet
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
data=load_diabetes()
x=data.data
y=data.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
ElasticNet=ElasticNet(alpha=1,l1_ratio=0.5)
ElasticNet.fit(x_train,y_train)
y_pred=ElasticNet.predict(x_test)
r2_score(y_test,y_pred)
```

## ✓ Logistic Regression

```
#works on where the output is lineary/nearly lineary seperable
#equation of the form  $ax_1+bx_2+c=0$ 
 $z=ax_1+bx_2+c$ 
#sigmoid(z)= $1/(1+e^{-z})$  → [0,1]

#perceptron trick can be used
#here sigmoid activation functions give probability of positive region/outcome , it creates a prob
#mostly used for binary classification with sigmoid
#for multiple , use ✅ Categorical Cross-Entropy with softmax activation function
#maximum likelihood gives best model from set of models (cross entropy is negative of probability)
#cross entropy increases , goodness of model decreases
from sklearn.linear_model import LogisticRegression
lr=LogisticRegression()
lr.fit(x_train,y_train)
lr.predict(x_test)
```

## ✓ Support Vector Machines (SVM)

```

##based on idea of logistic regression
#line parallel to hyperplane are support vectors
#works on higher dimensional data/non linear using kernel
#uses hinge loss
#robust to outliers
#line eqn->wx+b>=0 then positive class else negative class
#d=2/mod(w)
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
data=load_iris()
x_train,x_test,y_train,y_test=train_test_split(data.data,data.target,test_size=0.2,random_state=
svc=SVC(kernel='linear') #use linear kernel for linearly separable data
                        #use rbf/poly for non linear data (converts low dim to high dim data u
                        #c is a hyperparameter(regularisation term)

svc.fit(x_train,y_train)
y_pred=svc.predict(x_test)
accuracy_score(y_test,y_pred)

```

1.0

## ✓ classification Metrics

```

from sklearn.metrics import accuracy_score,confusion_matrix,f1_score,precision_score,recall_score
#accuracy=no of correct pred/total no of pred
#accuracy is misleading for imbalanced datasets
#use confusion matrix instead
#accuracy=(true positive + true negative)/(true positive+true negative + false positive+ false ne
#confusion matrix ->
#
#      |actual  1      0
#      |
#      1      tp      fn
#      0      fp      tn
#
#type 1 error=>fp
#type 2 error->fn
#good models have less false positives
#precision=true positive/true positive+false positive
#recall->true positive/true positive+false negative
#it is important to identify which type of error is matter for the use case
#f1 score= 2*precision*recall/precision+recall f1 score penalises lower val
as=accuracy_score(y_test,y_pred)
cm=confusion_matrix(y_test,y_pred)
f1=f1_score(y_test,y_pred)
ps=precision_score(y_test,y_pred)
rs=recall_score(y_test,y_pred)
cr=classification_report(y_test,y_pred)

```

## ✓ Softmax Regression /multinomial logistics regression

```

#use when output classes >2
#softmax gives probability for all the classes
#here class having high probability is the considered the output
#use sparse or categorical cross entropy loss

```

```

from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.metrics import f1_score, accuracy_score
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
data=load_iris()
le=LabelEncoder()
x=data.data
y=data.target
# y=y.reshape(-1,1)
# le=LabelEncoder()
# y=le.fit_transform(y)
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
lr=LogisticRegression(multi_class='multinomial')
lr.fit(x_train,y_train)
y_pred=lr.predict(x_test)#it return class having highest probbability
# f1=f1_score(y_test,y_pred,average='macro')
# ac=accuracy_score(y_test,y_pred)
y_pred

```

## ✓ Polynomial Logistics Regression

```

#USED WHEN DATA IS NOT LINEARly seperable
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
pr= PolynomialFeatures(degree=2) #here degree is an hyperparamter
x_train_transformed=pr.fit_transform(x_train)
x_test_transformed=pr.transform(x_test)
lr=LogisticRegression()
lr.fit(x_train_transformed,y_train)
lr.predict(x_test_transformed)

```

## ✓ Decision Tree Regressor and Classifier

```

#they are prone to overfitting
#logiric time complexity of class prediction
#entropy->  $e = -p_1 \log(p_1) - p_2 \log(p_2) - p_3 \log(p_3) - p_4 \log(p_4) - p_5 \log(p_5)$  ,,...whrere pi represnt of
#gini impirity= $1-(p_1^2+p_2^2+...)$ 
#less entropy , easy to predict
#information gain is the measure used for splitting criterria
#information gian-> $\text{entropy}(\text{parent}) - \text{weighted\_avg} * \text{entropy}(\text{children})$ 
#for children of parent nide are the possible splitts
#node with max information gain is selected as splitter
#high depth of tree->overfitting chances
#low depth ->underfiitting chances
#spliitter can be best or random
#for numerical input col , splits are tried between mid points of two consective val
#lr.feature_importances_ can be used for feature selection, it is based on how many times the co
#output label is one hot encoded
#cart alogrith is used for gini impirity
#ide alog usef for entroy
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.datasets import load_iris

```

```

from sklearn.metrics import f1_score, accuracy_score
from sklearn.model_selection import train_test_split
data = load_iris()
x = data.data
y = data.target
y.shape
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
lr = DecisionTreeClassifier(criterion="gini", max_depth=None)
# other important hyperparameter are min sample split etc
lr.fit(x_train, y_train)
y_pred = lr.predict(x_test)
f1 = f1_score(y_test, y_pred, average='macro')
ac = accuracy_score(y_test, y_pred)
# lr.get_depth()
# lr.get_params()
# lr.feature_importances_
plot_tree(lr)

```

## ✓ K nearest neighbours(KNN)

```

# distance based algo
# no of neighbours is an hyperparameter derived thorough cross val in gridsearchcv
# if k is low -> overfitting
# if k is high -> underfitting
# prone to outliers
# data needs to be scaled
# does not work well on high dimension data (due to curse of dimensionality)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder, StandardScaler
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from matplotlib import pyplot as plt

le = LabelEncoder()
sc = StandardScaler()

df = pd.read_csv("train.csv")
# df.head()
df = df.drop(columns=["PassengerId", "Name"], axis=1)
# df.head()
df = df.dropna()
df.head()
x = df.drop(columns=["Survived"])
y = df["Survived"]
x["sex"] = le.fit_transform(x["Sex"])
x = x.drop(columns=["Sex"])
x["Cabin"] = le.fit_transform(x["Cabin"])
x = x.drop(columns=["Cabin"])
x["Ticket"] = le.fit_transform(x["Ticket"])
# x = x.drop(columns=["Ticket"])
x["Embarked"] = le.fit_transform(x["Embarked"])
# x = x.drop(columns=["Embarked"])
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
x_train_scaled = sc.fit_transform(x_train)
x_test_scaled = sc.transform(x_test)

```

```

x_train_scaled
knn=KNeighborsClassifier()
y_preds=[]
for i in range(1,10):
    knn=KNeighborsClassifier(n_neighbors=i)
    knn.fit(x_train_scaled,y_train)
    y_pred=knn.predict(x_test_scaled)
    print(accuracy_score(y_test,y_pred))
    y_preds.append(accuracy_score(y_test,y_pred))

#df.shape
plt.plot(range(1,10),y_preds)

```

## ✓ Ensemble Learning

```

#ensemble learning(used for performance improvement,reduce bias and variance)
#output ->for classification->majority
#         ->for regression->mean Value of all the output

#types->1.voting
#       2.bagging
#       3.boosting
#       4.stacking

```

## ✓ Voting Classifier

```

#here all models are different from each other or same models with different configuration
#data provided is same
#all models should accuracy greater than 51%
#collective accuracy increases after applying voting
from sklearn.ensemble import VotingClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import f1_score,accuracy_score
data=load_iris()
x=data.data
y=data.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
lr=LogisticRegression()
dt=DecisionTreeClassifier()
svc=SVC()
vc=VotingClassifier(estimators=[('lr',lr),('dt',dt),('svc',svc)])
vc.fit(x_train,y_train)
y_pred=vc.predict(x_test)
f1=f1_score(y_test,y_pred,average='macro')
#vc.estimators_

```

## Voting Regressor

```
from sklearn.ensemble import VotingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_diabetes
from sklearn.metrics import r2_score, root_mean_squared_error
data=load_diabetes()
x=data.data
y=data.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
lr=LinearRegression()
dt=DecisionTreeRegressor()
svc=SVR()
vc=VotingRegressor(estimators=[('lr',lr),('dt',dt),('svc',svc)])
vc.fit(x_train,y_train)
y_pred=vc.predict(x_test)
r2=r2_score(y_test,y_pred)
#r2=mean_squared_error(y_test,y_pred)
r2
```

## Bagging Classifier

```
#here use same model with diffrer data(i.e bootstrapping)
#eg.random forest
#generally use low bais and high variance models
#bagging smotthens descion boundary
#pasting->row samplign withour replacement
from sklearn.ensemble import BaggingClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
data=load_iris()
x=data.data
y=data.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=2)
lr=LogisticRegression(penalty="l2")
bc=BaggingClassifier(estimator=lr,n_estimators=3,max_samples=100,max_features=0.5)#max-samples->
bc.fit(x_train,y_train)
y_pred=bc.predict(x_test)
f1=f1_score(y_test,y_pred,average='macro')
f1
```

0.9581699346405229

## Baggin Regressor

```
#oob ->out of bag samples->testing model on points which are not given to any model
#around 37% are suhc points
```

```

from sklearn.ensemble import BaggingRegressor
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
data=load_diabetes()
x=data.data
y=data.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=2)
lr=LinearRegression()
br=BaggingRegressor(estimator=lr,n_estimators=10,max_samples=150,max_features=1.0,oob_score=True)
br.fit(x_train,y_train)
y_pred=br.predict(x_test)
r2=r2_score(y_test,y_pred)
#br.oob_score_

```

## ✓ Random Forest classifier

```

#here base estimators is decision tree
#it is better than normal bagging because it includes more randomness while training
#row and col sampling(on node level) is done
#it used for to calculate feature importances
#feature importance[i]=impurity[i]-left node impurity[i]-right node impurity[i]
#hyperparameters same as that of decision tree classifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.tree import plot_tree
from sklearn.datasets import load_iris
data=load_iris()
x=data.data
y=data.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=2)
rf=RandomForestClassifier(n_estimators=100)
rf.fit(x_train,y_train)
y_pred=rf.predict(x_test)
y_pred

```

## ✓ Random Forest Regressor

```

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.tree import plot_tree
from sklearn.datasets import load_diabetes
data=load_diabetes()
x=data.data
y=data.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=2)
rf=RandomForestRegressor(n_estimators=100,max_features=0.8)
rf.fit(x_train,y_train)

```



```
y_pred=rf.predict(x_test)
```

## ✓ K means clustering

```
#its unsupervised learning algorithm
# #steps->1.define no of clisters
#         2.init conetroids
#         3.put all the points to respective cluster using euclideran
#         4.calucate new centroid
#         5.if prev centroud ==new cneroid stop else continue
#no of cluster can be found using wcss (elbow mehtod)
from sklearn.cluster import KMeans
import pandas as pd
import matplotlib.pyplot as plt
df=pd.read_csv("student_clustering.csv")
wcss=[]
for i in range(1,15):
    km=KMeans(n_clusters=i)
    km.fit_predict(df)
    wcss.append(km.inertia_)

plt.plot(range(1,15),wcss)
#chose no of cluster whree line becoomes straight,here its pt 4
km=KMeans(n_clusters=4)
y=km.fit_predict(df)
```

## ✓ DBSCAN(density based spatial clustering of aplication with noise)